Principios de programación segura

1. Validación y gestión de errores

Objetivo de la sesión

Controlar lo que entra y lo que sale de una API

ENTRADA

Validar antes de ejecutar la lógica de negocio

ERRORES

Convertir excepciones en respuestas coherentes

SEGURIDAD

No filtrar detalles internos al cliente

Idea clave: una API segura no confía ciegamente en sus entradas ni improvisa sus errores.

¿Qué significa que una aplicación sea segura?

Tres propiedades que debemos proteger

Confidencialidad

Que nadie lea
lo que no debe.

Integridad

Que nadie modifique
lo que no debe.

Disponibilidad

Que el servicio siga
funcionando.

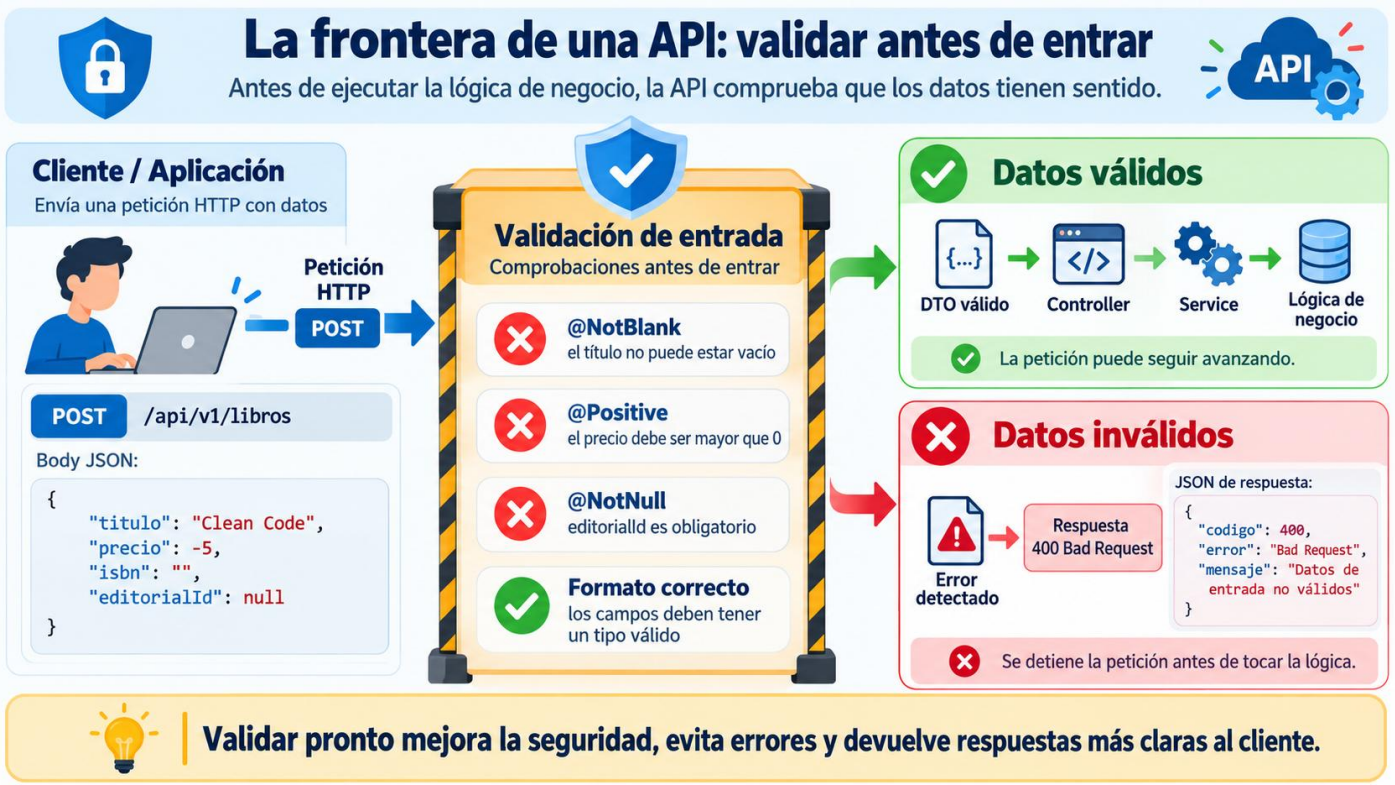
La seguridad no empieza en el login: empieza en cada punto donde algo externo puede interactuar con el sistema.

Superficie de ataque = endpoints + datos de entrada + errores + credenciales + configuración expuesta

Superficie de ataque y defensa en profundidad



Validar en la frontera



En código

```
@NotBlank(message = "El título
no puede estar vacío")
@PositiveOrZero(message = "El
precio no puede ser negativo")

@PostMapping
create(@Valid @RequestBody
LibroCreateDTO dto)
```

Si el DTO no cumple las reglas, la petición se detiene antes de entrar en la lógica de negocio.

Un formato común para todos los errores

Anatomía de ErrorResponse

Un formato uniforme para que el cliente entienda qué ha fallado sin ver detalles internos.

Petición que falla

POST /api/v1/libros

Body JSON

```
{
  "titulo": "",
  "precio": -5,
  "editorialId": null
}
```

Bean Validation detecta
campos inválidos antes de ejecutar la lógica de negocio.

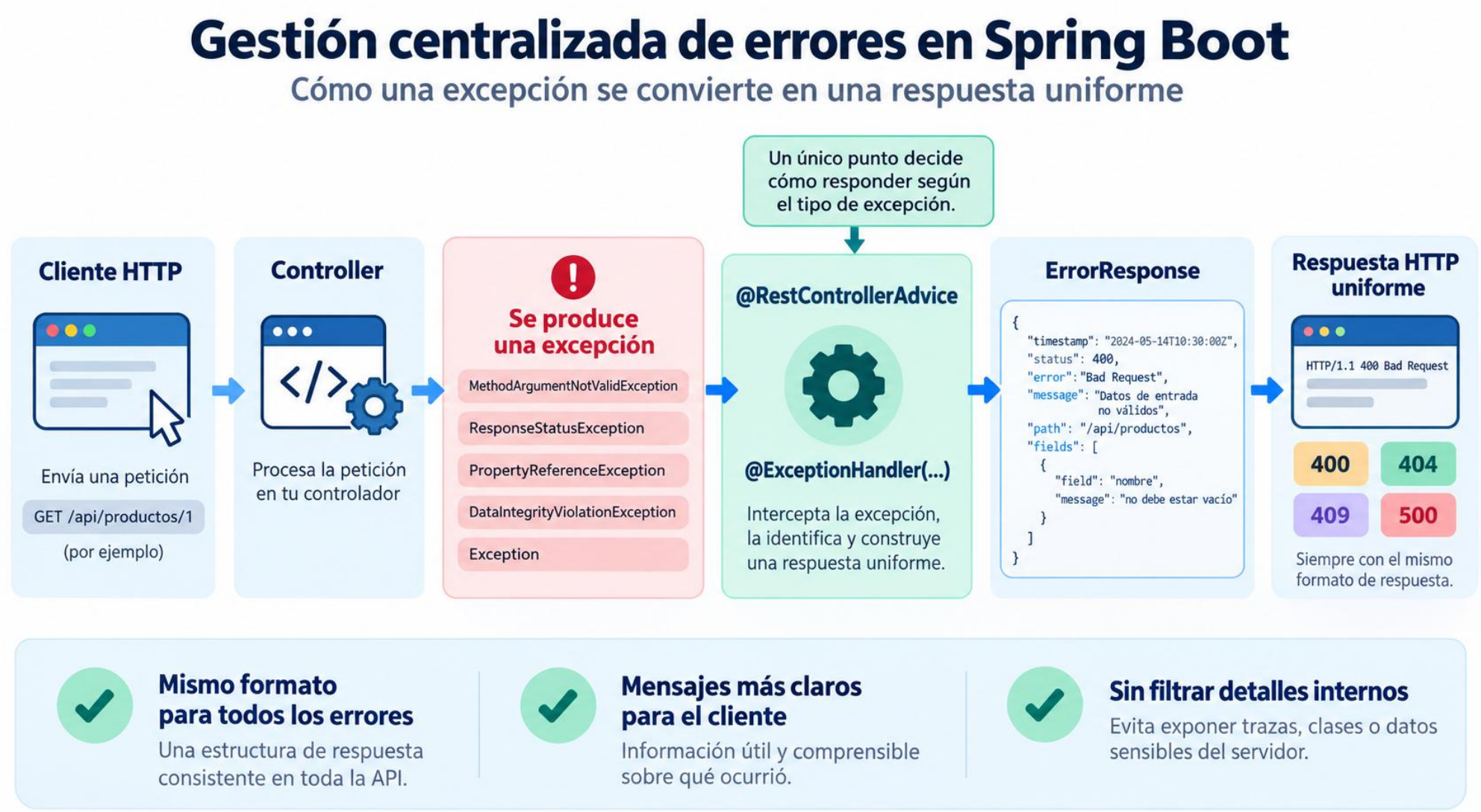
Respuesta HTTP uniforme

```
{
  "timestamp": "2026-10-20T14:04:31",
  "status": 400,
  "error": "Error de validación",
  "message": "La petición contiene campos inválidos",
  "path": "/api/v1/libros",
  "fields": {
    "titulo": ["El título no puede estar vacío"],
    "precio": ["El precio no puede ser negativo"],
    "editorialId": ["La editorial es obligatoria"]
  }
}
```

- `timestamp` — **Cuándo ocurrió**: Marca el instante del error.
- `status` — **Código HTTP**: 400, 404, 409, 500...
- `error` — **Nombre corto**: Categoría legible del problema.
- `message` — **Mensaje para el cliente**: Claro, útil y sin detalles internos.
- `path` — **Ruta afectada**: Qué endpoint estaba atendiendo la petición.
- `fields` — **Errores por campo**: Solo en validaciones con campos concretos.

Idea clave | **El cliente recibe información útil y estable; los detalles técnicos se quedan en los logs del servidor.**

Centralizar la gestión de errores



Handler 1 — errores de validación

```
@ExceptionHandler(MethodArgumentNotValidExcept
ion.class)
public ResponseEntity<ErrorResponse>
handleValidationExceptions(...) {

  Map<String, List<String>> fields =
      ex.getBindingResult().getFieldErrors()
        .stream()
        .collect(...);

  return ResponseEntity
      .status(HttpStatus.BAD_REQUEST)
      .body(response);
}
```

¿Qué hace?

- Recoge todos los campos que han fallado.
- Agrupa campo → lista de mensajes.
- Devuelve 400 Bad Request.
- Mantiene el mismo ErrorResponse que el resto de errores.

Cliente: qué campo ha fallado y por qué.

No todo error es un 500

Clasificar bien el problema produce respuestas más útiles

404	`ResponseStatusException`	**Recurso inexistente**
400	`PropertyReferenceException`	**Parámetro de consulta inválido**
409	`DataIntegrityViolationException`	**Conflicto con el estado de los datos**

Pregunta útil: ¿el cliente ha enviado algo incorrecto, no existe el recurso o la operación entra en conflicto con los datos?

Handler 5 — la red de seguridad final

```
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorResponse>
handleGenericException(...) {

  ErrorResponse response =
      new ErrorResponse(
          ..., 500,
          "Error interno",
          "Ha ocurrido un error inesperado",
          request.getRequestURI());

  return ResponseEntity
.status(HttpStatus.INTERNAL_SERVER_ERROR)
      .body(response);
}
```

NO devolver ex.getMessage()

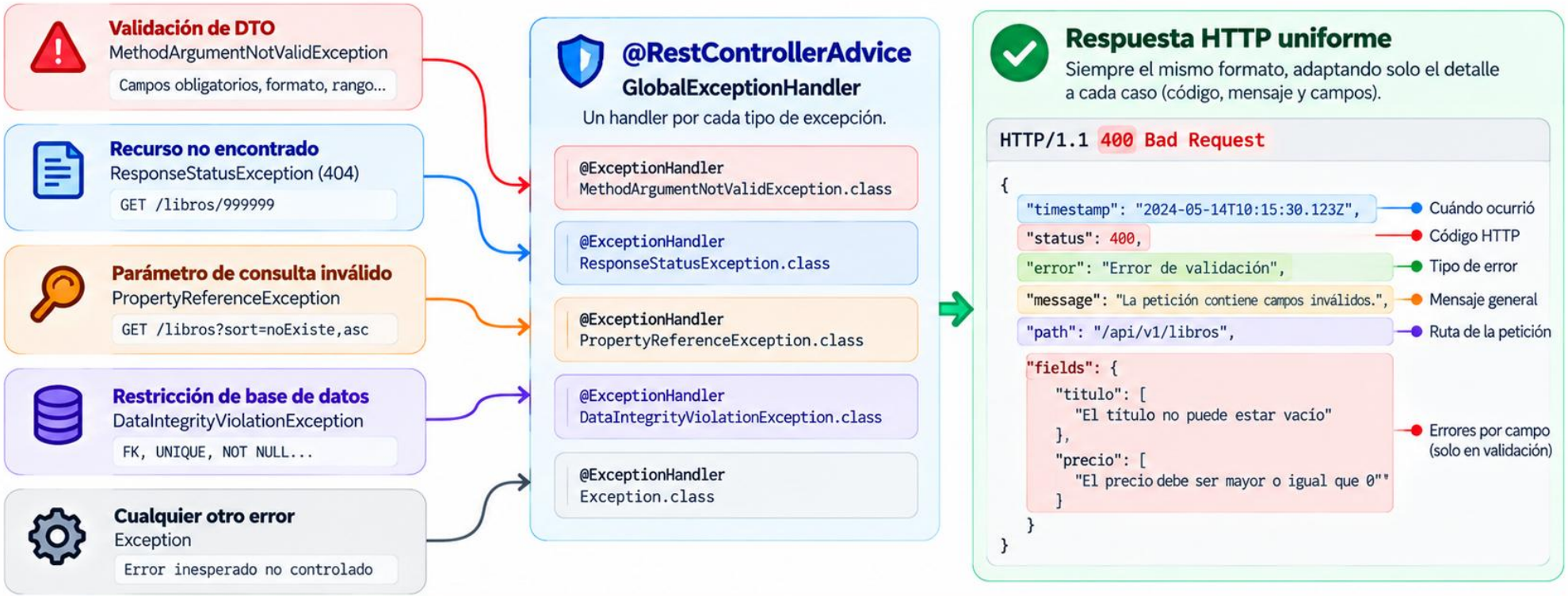
- Puede revelar clases, tablas o rutas internas.
- El cliente necesita un mensaje seguro y estable.
- La excepción completa sí debe quedar en los logs del servidor.

Respuesta pública ≠ información de diagnóstico

Cinco errores, cinco handlers

4. De la excepción a una respuesta uniforme

Un @RestControllerAdvice centraliza la gestión de errores. Cada tipo de excepción se captura con un handler específico y se convierte en el mismo formato de respuesta, sin filtrar información interna.



Ventajas de una gestión centralizada de errores

Formato coherente
Todas las respuestas de error siguen el mismo esquema.

Mensajes útiles para el cliente
Indican qué ha fallado sin revelar detalles internos.

Sin filtrado de información
Evita exponer trazas, nombres de tablas o rutas del servidor.

Fácil de mantener
Los cambios se hacen en un único lugar, sin duplicar lógica en cada controller.

Ahora lo aplicas en GameVault

La teoría usa una API genérica; la práctica exige transferir las mismas decisiones a tu proyecto.

1

Provocar

Observar cinco errores antes de gestionarlos

2

Centralizar

Construir ErrorResponse + GlobalExceptionHandler

3

Repetir

Lanzar exactamente los mismos errores

4

Comparar

Código HTTP, formato y datos expuestos

Objetivo: que un mismo problema produzca una respuesta coherente, comprensible y segura.

Ideas clave

- Una API expuesta tiene una superficie de ataque: no se confía en ningún dato externo hasta validarlo.
- @Valid y Bean Validation rechazan entradas incorrectas antes de ejecutar la lógica de negocio.
- ErrorResponse da un contrato común a los errores de la API.
- @RestControllerAdvice + @ExceptionHandler centralizan la conversión de excepciones en respuestas HTTP.
- 400, 404, 409 y 500 expresan problemas distintos: clasificar bien el error importa.
- El handler genérico evita respuestas inconsistentes, pero no debe filtrar detalles internos.

Siguiente paso: Spring Security — autenticación, autorización y una primera política de acceso.